

LICENCIATURA EM INFORMÁTICA E GESTÃO DE EMPRESAS

iscte

INSTITUTO
UNIVERSITÁRIO
DE LISBOA

Projeto de Base de Dados: **SIEstágios** PARTE 2



Grupo 26

Rodrigo Vicente nº129633
Henrique Martins nº132007
Pedro Azevedo nº130659
Bernardo Villa nº124197

Docente: Prof. António Galguinho
Data: 21/12/2025

Índice:

Introdução	3
1. Automatismos	
1.1 Triggers	4
1.2 Stored Procedures	5
1.3 Funções	7
2. Consultas SQL-DML e Views	9
Conclusão	11

INTRODUÇÃO

O presente relatório descreve o trabalho feito na Parte 2 do projeto SIEstágios, cujo objetivo é apoiar a gestão do processo de estágio (registo, acompanhamento e avaliação) através de uma base de dados relacional. Nesta fase, o objetivo foi pegar no modelo da Parte 1 e acrescentar componentes que ajudam a manter os dados consistentes e a obter informação de forma mais prática.

Em concreto, foram implementados automatismos na base de dados (triggers, stored procedures e funções) e criadas consultas SQL-DML e views para responder aos pedidos de informação definidos. Além disso, foi desenvolvido o protótipo web, ligado à base de dados, que permite executar as operações previstas.

1. Automatismos

Nesta fase foram implementados automatismos na base de dados para reforçar a integridade e garantir que regras essenciais são aplicadas diretamente no SGBD. Desta forma, validações e operações recorrentes ficam centralizadas, reduzindo incoerências e evitando dependência exclusiva da aplicação.

1.1. Triggers

Os triggers permitem validar automaticamente condições no momento em que ocorre uma operação de escrita. No projeto, foram aplicadas à tabela estagio para impedir a gravação de registos com valores inválidos, assegurando consistência desde a inserção.

T1- Impedir a inserção de estágios com classificacao fora do intervalo permitido (1 a 5).

Nome do trigger	T1
Tabela	estagio
Tempo	BEFORE
Evento	INSERT
Definição	<pre>1 BEGIN 2 IF NEW.Classificacao < 1 OR NEW.Classificacao > 5 THEN 3 SIGNAL SQLSTATE '45000' 4 SET MESSAGE_TEXT = 'Classificação do estagiário deve estar entre 1 e 5'; 5 END IF; 6 END</pre>
Definidor	root@%

T2- Impedir a inserção de estágios em que a data de inicio seja posterior a data do fim.

Nome do trigger	T2
Tabela	estagio
Tempo	BEFORE
Evento	INSERT
Definição	<pre>1 IF NEW.data_inicio > NEW.data_fim THEN 2 SIGNAL SQLSTATE '45000' 3 SET MESSAGE_TEXT = 'A data de início não pode ser posterior à data de fim'; 4 END IF</pre>
Definidor	root@%

1.2. Stored Procedures

As stored procedures foram usadas para encapsular operações que exigem validações e várias etapas, garantindo que a execução segue sempre a mesma lógica. Isto aumenta a reutilização, melhora a organização do código e permite devolver erros claros quando as condições necessárias não são cumpridas.

P1- Validar a existência de aluno, formador e estabelecimento antes de inserir um estágio.

Nome da rotina	P1																																																							
Tipo	PROCEDURE Alterar para FUNCTION																																																							
Parâmetros	<table><thead><tr><th>Direção</th><th>Nome</th><th>Tipo</th><th>Tamanho/Valores*</th><th>Opções</th></tr></thead><tbody><tr><td>‡ IN</td><td> p_aluno_id </td><td> INT </td><td> </td><td>Mapa de Caractere</td></tr><tr><td>‡ IN</td><td> p_empresa_id </td><td> INT </td><td> </td><td>Mapa de Caractere</td></tr><tr><td>‡ IN</td><td> p_estabelecimento_id </td><td> INT </td><td> </td><td>Mapa de Caractere</td></tr><tr><td>‡ IN</td><td> p_formador_id </td><td> INT </td><td> </td><td>Mapa de Caractere</td></tr><tr><td>‡ IN</td><td> p_data_inicio </td><td> DATE </td><td> </td><td>Mapa de Caractere</td></tr><tr><td>‡ IN</td><td> p_data_fim </td><td> DATE </td><td> </td><td>Mapa de Caractere</td></tr><tr><td>‡ IN</td><td> p_nota_escola </td><td> INT </td><td> </td><td>Mapa de Caractere</td></tr><tr><td>‡ IN</td><td> p_nota_relatorio </td><td> INT </td><td> </td><td>Mapa de Caractere</td></tr><tr><td>‡ IN</td><td> p_nota_procura </td><td> INT </td><td> </td><td>Mapa de Caractere</td></tr><tr><td>‡ IN</td><td> p_classificacao </td><td> TINYINT </td><td> </td><td>Mapa de Caractere</td></tr></tbody></table>	Direção	Nome	Tipo	Tamanho/Valores*	Opções	‡ IN	p_aluno_id	INT		Mapa de Caractere	‡ IN	p_empresa_id	INT		Mapa de Caractere	‡ IN	p_estabelecimento_id	INT		Mapa de Caractere	‡ IN	p_formador_id	INT		Mapa de Caractere	‡ IN	p_data_inicio	DATE		Mapa de Caractere	‡ IN	p_data_fim	DATE		Mapa de Caractere	‡ IN	p_nota_escola	INT		Mapa de Caractere	‡ IN	p_nota_relatorio	INT		Mapa de Caractere	‡ IN	p_nota_procura	INT		Mapa de Caractere	‡ IN	p_classificacao	TINYINT		Mapa de Caractere
Direção	Nome	Tipo	Tamanho/Valores*	Opções																																																				
‡ IN	p_aluno_id	INT		Mapa de Caractere																																																				
‡ IN	p_empresa_id	INT		Mapa de Caractere																																																				
‡ IN	p_estabelecimento_id	INT		Mapa de Caractere																																																				
‡ IN	p_formador_id	INT		Mapa de Caractere																																																				
‡ IN	p_data_inicio	DATE		Mapa de Caractere																																																				
‡ IN	p_data_fim	DATE		Mapa de Caractere																																																				
‡ IN	p_nota_escola	INT		Mapa de Caractere																																																				
‡ IN	p_nota_relatorio	INT		Mapa de Caractere																																																				
‡ IN	p_nota_procura	INT		Mapa de Caractere																																																				
‡ IN	p_classificacao	TINYINT		Mapa de Caractere																																																				

Código:

BEGIN

```
IF NOT EXISTS (  
    SELECT 1 FROM aluno  
    WHERE utilizador_id = p_aluno_id  
) THEN  
    SIGNAL SQLSTATE '45000'  
    SET MESSAGE_TEXT = 'Aluno não existe.';  
END IF;
```

```
IF NOT EXISTS (  
    SELECT 1 FROM formador  
    WHERE utilizador_id = p_formador_id  
) THEN  
    SIGNAL SQLSTATE '45000'  
    SET MESSAGE_TEXT = 'Formador não existe.';  
END IF;
```

```
IF NOT EXISTS (  
    SELECT 1 FROM estabelecimento  
    WHERE empresa_id = p_empresa_id  
    AND estabelecimento_id = p_estabelecimento_id  
) THEN  
    SIGNAL SQLSTATE '45000'  
    SET MESSAGE_TEXT = 'Estabelecimento não existe.';  
END IF;
```

```
INSERT INTO estagio (  
    estabelecimento_empresa_id,  
    estabelecimento_id,  
    aluno_id,  
    formador_id,  
    data_inicio,  
    data_fim,  
    nota_escola,  
    nota_relatorio,  
    nota_procura,  
    classificacao
```

```
) VALUES (  
    p_empresa_id,  
    p_estabelecimento_id,  
    p_aluno_id,  
    p_formador_id,  
    p_data_inicio,  
    p_data_fim,  
    p_nota_escola,  
    p_nota_relatorio,  
    p_nota_procura,  
    p_classificacao
```

```
);
```

END

P2- Devolver os estágios cuja data de início ocorre entre hoje e hoje + N dias.

Nome da rotina: P2

Tipo: PROCEDURE

Alterar para FUNCTION

Direção	Nome	Tipo	Tamanho/Valores*	Opções
IN	p_dias	INT		Mapa de Caractere

Adicionar parâmetro Remover último parâmetro

Definição

```
BEGIN
  IF p_dias < 0 THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'O número de
dias não pode ser negativo.';
  END IF;

  SELECT *
  FROM estagio
  WHERE data_inicio BETWEEN CURDATE()
    AND
    DATE_ADD(CURDATE(), INTERVAL p_dias DAY)
  ORDER BY data_inicio;
END
```

É determinístico: ☐

Ajustar privilégios: ☒

Definidor: 'root'@'localhost'

Tipo de segurança: DEFINER

Acesso de dados SQL: CONTAINS SQL

1.3. Funções

As funções permitem calcular valores derivados de forma consistente e reutilizável dentro da base de dados. No projeto, foram utilizadas para apurar métricas relacionadas com avaliação do estágio, evitando repetição de cálculos e facilitando a integração em consultas e relatórios.

F1- Calcular a média de classificação de um estabelecimento num determinado ano letivo

Nome da rotina: F1

Tipo: FUNCTION

Alterar para PROCEDURE

Nome	Tipo	Tamanho/Valores*	Opções
p_estabelecimento_empresa_id	INT		Mapa de Caractere
p_estabelecimento_id	INT		Mapa de Caractere
p_ano_letivo	INT		Mapa de Caractere

Adicionar parâmetro Remover último parâmetro

Tipo de retorno: DECIMAL

Retornar comprimento/valores: 5,2

Opções de retorno: Mapa de Caractere

Definição

```
BEGIN
  DECLARE v_media DECIMAL(5,2);

  SELECT AVG(classificacao)
  INTO v_media
  FROM estagio
  WHERE estabelecimento_empresa_id =
p_estabelecimento_empresa_id
    AND estabelecimento_id =
p_estabelecimento_id
    AND classificacao IS NOT NULL
    AND data_inicio >=
CONCAT(p_ano_letivo, '-09-01')
    AND data_inicio <
CONCAT(p_ano_letivo + 1, '-09-01');

  RETURN v_media;
END
```

É determinístico: ☒

Ajustar privilégios: ☒

Definidor: 'root'@'localhost'

Tipo de segurança: DEFINER

Código:

```
BEGIN
  DECLARE v_media DECIMAL(5,2);

  SELECT AVG(classificacao)
  INTO v_media
  FROM estagio
  WHERE estabelecimento_empresa_id = p_estabelecimento_empresa_id
    AND estabelecimento_id = p_estabelecimento_id
    AND classificacao IS NOT NULL
    AND data_inicio >= CONCAT(p_ano_letivo, '-09-01')
    AND data_inicio < CONCAT(p_ano_letivo + 1, '-09-01');

  RETURN v_media;
END
```

F2- Cálculo de média ponderada de notas do estágio

Nome da rotina: F2

Tipo: **FUNCTION**
 Alterar para PROCEDURE

Nome	Tipo	Tamanho/Valores*	Opções
p_estabelecimento_empres	INT		Mapa de Caractere
p_estabelecimento_id	INT		Mapa de Caractere
p_aluno_id	INT		Mapa de Caractere
p_peso_empresa	DECIMAL	10,4	Mapa de Caractere
p_peso_escola	DECIMAL	10,4	Mapa de Caractere
p_peso_relatorio	DECIMAL	10,4	Mapa de Caractere
p_peso_procura	DECIMAL	10,4	Mapa de Caractere

Adicionar parâmetro Remover último parâmetro

Tipo de retorno: DECIMAL

Retornar comprimento/valores: 5,2

Opções de retorno: Mapa de Caractere

É determinístico: ☒

Ajustar privilégios: ☒

Definidor: 'root'@'localhost'

Tipo de segurança: DEFINER

Acesso de dados SQL: CONTAINS SQL

Código:

BEGIN

```

DECLARE v_empresa DOUBLE;
DECLARE v_escola DOUBLE;
DECLARE v_relatorio DOUBLE;
DECLARE v_procura DOUBLE;
DECLARE v_soma DECIMAL(20,8);
DECLARE v_res DECIMAL(10,4);

```

```

IF NOT EXISTS (
    SELECT 1
    FROM estagio
    WHERE estabelecimento_empresa_id = p_estabelecimento_empresa_id
    AND estabelecimento_id = p_estabelecimento_id
    AND aluno_id = p_aluno_id
) THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Estágio não existe.';
END IF;

```

```

    SET v_soma = p_peso_empresa + p_peso_escola + p_peso_relatorio +
p_peso_procura;

```

```

IF v_soma = 0 THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Soma dos pesos não pode ser 0.';
END IF;

```



```

SELECT nota_empresa, nota_escola, nota_relatorio, nota_procura
      INTO v_empresa, v_escola, v_relatorio, v_procura
FROM estagio
WHERE estabelecimento_empresa_id = p_estabelecimento_empresa_id
      AND estabelecimento_id = p_estabelecimento_id
      AND aluno_id = p_aluno_id
LIMIT 1;

SET v_res =
  (v_empresa * p_peso_empresa +
   v_escola * p_peso_escola +
   v_relatorio * p_peso_relatorio +
   v_procura * p_peso_procura) / v_soma;

RETURN ROUND(v_res, 2);
END

```

2. Consultas SQL-DML e Views

As funções permitem calcular valores derivados de forma consistente e reutilizável dentro da base de dados. No projeto, foram utilizadas para apurar métricas relacionadas com avaliação do estágio, evitando repetição de cálculos e facilitando a integração em consultas e relatórios.

Q1- O nome do formador e o número total de estágios orientados, apenas quando esse total é superior a 1.

Código:

```

select `u`.`nome` AS `nome_formador`, count(0) AS `total_estagios` from
((`siestagios`.`estagio` `e` join `siestagios`.`formador` `f` on(`f`.`utilizador_id` =
`e`.`formador_id`)) join `siestagios`.`utilizador` `u` on(`u`.`utilizador_id` =
`f`.`utilizador_id`)) group by `u`.`utilizador_id`, `u`.`nome` having count(0) > 1

```

nome_formador	total_estagios
Ana Mendes	4
Ricardo Gomes	2

Q2- O nome da empresa e a média das notas atribuídas aos alunos, apenas para empresas com média igual ou superior a 14.

Código:

```

select `e`.`firma` AS `nome_empresa`, avg(`s`.`nota_empresa`) AS `media_nota_empresa`
from (`siestagios`.`empresa` `e` join `siestagios`.`estagio` `s`
on(`s`.`estabelecimento_empresa_id` = `e`.`empresa_id`)) group by
`e`.`empresa_id`, `e`.`firma` having avg(`s`.`nota_empresa`) >= 14

```

nome_empresa	media_nota_empresa
TecSoft	16
MarketPlus	14
ContabPro	19

Q3- Empresas e total de produtos comercializados,para empresas que comercializem pelo menos 1 produto

Código:

```
select `e`.`firma` AS `nome_empresa`,count(`c`.`produto_id`) AS `total_produtos` from
(`siestagios`.`empresa` `e` join `siestagios`.`comercializa` `c`
on(`c`.`estabelecimento_empresa_id` = `e`.`empresa_id`)) group by `e`.`firma` having
count(`c`.`produto_id`) >= 1
```

nome_empresa	total_produtos
ContabPro	1
MarketPlus	1
TecSoft	1

Q4- Empresas e o número total de estágios associados, apenas quando existe pelo menos um estágio.

Código:

```
select `e`.`firma` AS `nome_empresa`,count(0) AS `total_estagios` from
(`siestagios`.`empresa` `e` join `siestagios`.`estagio` `s`
on(`s`.`estabelecimento_empresa_id` = `e`.`empresa_id`)) group by
`e`.`empresa_id`,`e`.`firma` having count(0) >= 1
```

nome_empresa	total_estagios
TecSoft	3
MarketPlus	1
ContabPro	2

Q5- Cursos com número de turmas superior à média de turmas por curso.

Código:

```
select `c`.`designacao` AS `nome_curso`,count(`t`.`turma_id`) AS `total_turmas` from
(`siestagios`.`curso` `c` join `siestagios`.`turma` `t` on(`t`.`curso_id` = `c`.`curso_id`))
group by `c`.`curso_id`,`c`.`designacao` having count(`t`.`turma_id`) > (select
avg(`sub`.`num_turmas`) from (select count(0) AS `num_turmas` from `siestagios`.`turma`
group by `siestagios`.`turma`.`curso_id`) `sub`)
```

nome_curso	total_turmas
Informática	2

Q6- Formadores cuja média de nota final seja superior à média global das notas finais

Código:

```
select `u`.`nome` AS `nome_formador`,avg(`e`.`nota_final`) AS `media_formador`,(select
avg(`siestagios`.`estagio`.`nota_final`) from `siestagios`.`estagio` where
`siestagios`.`estagio`.`nota_final` is not null) AS `media_global` from
((`siestagios`.`estagio` `e` join `siestagios`.`formador` `f` on(`f`.`utilizador_id` =
`e`.`formador_id`)) join `siestagios`.`utilizador` `u` on(`u`.`utilizador_id` =
`f`.`utilizador_id`)) where `e`.`nota_final` is not null group by `u`.`nome` having
avg(`e`.`nota_final`) > (select avg(`siestagios`.`estagio`.`nota_final`) from
`siestagios`.`estagio` where `siestagios`.`estagio`.`nota_final` is not null)
```

nome_formador	media_formador	media_global
Ricardo Gomes	18	17

V1- Apresentar, por formador, o total de estágios orientados e a média da nota final, incluindo também a média global como referência.

Código:

```
select      `u`.`nome`          AS      `nome_formador`,count(`e`.`formador_id`)      AS
`total_estagios`,avg(`e`.`nota_final`)      AS      `media_notas_formador`,(select
avg(`siestagios`.`estagio`.`nota_final`)      from      `siestagios`.`estagio`      where
`siestagios`.`estagio`.`nota_final`      is      not      null)      AS      `media_global_notas`      from
((`siestagios`.`estagio`      `e`      join      `siestagios`.`formador`      `f`      on(`f`.`utilizador_id`      =
`e`.`formador_id`)))      join      `siestagios`.`utilizador`      `u`      on(`u`.`utilizador_id`      =
`f`.`utilizador_id`)) where `e`.`nota_final` is not null group by `u`.`nome`
```

nome_formador	total_estagios	media_notas_formador	media_global_notas
Ana Mendes	2	16	17
Ricardo Gomes	2	18	17

V2- Calcular a média de nota final por empresa e curso

Código:

```
select      `emp`.`firma`          AS      `nome_empresa`,`c`.`designacao`      AS
`nome_curso`,avg(`e`.`nota_final`)      AS      `media_nota_final`      from      (((`siestagios`.`estagio`
`e`      join      `siestagios`.`empresa`      `emp`      on(`emp`.`empresa_id`      =
`e`.`estabelecimento_empresa_id`))) join `siestagios`.`aluno`      `a`      on(`a`.`utilizador_id`      =
`e`.`aluno_id`)) join `siestagios`.`turma`      `t`      on(`t`.`turma_id`      =      `a`.`turma_id`)) join
`siestagios`.`curso`      `c`      on(`c`.`curso_id`      =      `t`.`curso_id`)) where `e`.`nota_final` is not
null group by `emp`.`firma`,`c`.`designacao`
```

nome_empresa	nome_curso	media_nota_final
ContabPro	Informática	18
MarketPlus	Informática	15
TecSoft	Informática	17

CONCLUSÃO

Com a realização da Parte 2, o projeto SIEstágios ficou concluído ao nível do que era pedido: a base de dados passou a incluir os automatismos definidos (triggers, stored procedures e funções) e foi criado o conjunto de consultas SQL-DML e views necessárias para obter a informação de forma organizada e reutilizável. Isto ajuda a manter os dados consistentes e facilita o acesso aos resultados, tanto para análise como para uso direto pela aplicação.

Na componente web, tivemos algumas dificuldades durante o desenvolvimento e na integração com a base de dados, mas no final o protótipo ficou completo e funcional. No geral, o trabalho permitiu consolidar a ligação entre a base de dados e a aplicação, e cumprir os objetivos principais desta fase do projeto.